Transformers Lab

🔰 Part 1: Introduction to Transformers

Objective: Understand the architecture and intuition behind Transformer models.

- 🔹 **Task 1.1: Self-Attention Mechanism**
  - **Goal:** Implement scaled dot-product attention with toy input.
  - **Hints:** Use numpy or PyTorch. Inputs: Q, K, V matrices.
  - **Questions:**
    - What happens to output when all attention weights are equal?
    - How does scaling affect the output?

- 🔹 **Task 1.2: Multi-Head Attention**
  - **Goal:** Simulate multi-head attention with random matrices.
  - **Hints:** Split Q, K, V into multiple heads, apply attention, then concatenate.
  - **Question:**
    - Why do we need multiple heads instead of one?

- 🔹 **Task 1.3: Positional Encoding**
  - **Goal:** Add sinusoidal positional encoding to a sequence.
  - **Example:** Apply positional encodings to a toy sequence of word vectors.
  - **Hint:** Use `sin(pos/10000^(2i/d))` formula.
  - **Question:**
    - Why is positional information required in Transformers?

🔁 Part 2: Building Transformer Components (Modular)

Objective: Recreate Transformer encoder layer manually using PyTorch.

- 🔹 **Task 2.1: Layer Normalization & Residual Connections**
  - Implement layer normalization.
  - Apply skip connection: `x + Sublayer(x)`.
  - **Question:** What role does residual learning play in convergence?

- 🔹 **Task 2.2: Feed Forward Networks**
  - Apply 2 linear layers with ReLU in between to word embeddings.
  - **Hint:** Use dimensions like 512 → 2048 → 512.

- 🔹 **Task 2.3: Assembling Encoder Block**
  - Chain Multi-Head Attention → Add & Norm → Feed Forward → Add & Norm.
  - Test block on a dummy sequence.

📦 Part 3: Pretrained Transformers (Applied)

Objective: Use pretrained transformer-based models for real-world tasks.

- 🔹 **Task 3.1: Text Classification using DistilBERT**
  - **Dataset:** AG News dataset
  - **Link:** https://www.kaggle.com/datasets/amananandrai/ag-news-classification-dataset
  - **Tools:** HuggingFace Transformers
  - **Hints:** Use `AutoTokenizer`, `AutoModelForSequenceClassification`, `Trainer`

- **Question:** How does transfer learning reduce training time?

🔹 **Task 3.2: Named Entity Recognition (NER)**

- **Dataset:** CoNLL-2003 (available via `datasets` library)
- Use any Transformer-based model (like BERT) with token classification head.
- Evaluate with `seqeval` metrics.

🚀 Part 4: Capstone Project

Project Title: Amazon Product Review Sentiment Analysis

- **Dataset:** https://www.kaggle.com/datasets/bittlingmayer/amazonreviews
- **Goal:** Use pretrained transformer (e.g., DistilBERT, RoBERTa) to classify reviews as positive or negative.
- **Steps:**
    - Load and preprocess data
    - Tokenize reviews
    - Train transformer model head for classification
    - Evaluate using accuracy and F1-score
- **Reflection Questions:**
    - What were the biggest gains using transformers vs RNN-based models?
    - What data preprocessing steps were crucial?
